



Sign In

- [Home](#) ›
- [Cortex XDR](#) ›
- [Cortex XDR 5.x Documentation](#) ›
- [Detect, investigate, and respond to threats](#) ›
- [Vulnerability management](#) ›
- [Investigate and remediate vulnerabilities](#) ›
- View vulnerability issues

Platform, XDR with Cloud

XDR + Cloud

Cortex XDR

11-05-2026



[Learn about Cortex XDR](#)

[Onboard Cortex XDR](#)

[Configure Cortex XDR](#)

[Protect your endpoints](#)

[Detect, investigate, and respond to threats](#)

[Monitor dashboards and reports](#)

[Asset management](#)

[Investigate and respond to cases](#)

[Agentic Assistant chat](#)
[Threat management](#)
[Attack Surface Management](#)
[Monitor and track compliance adherence](#)
[Cortex Advanced Email Security](#)
[Exposure management](#)
[Vulnerability management](#)
[Vulnerability management in Cortex XDR](#)
[Cortex Vulnerability Risk Score](#)
[Vulnerability policies](#)
[Investigate and remediate vulnerabilities](#)
[View all Vulnerabilities](#)
[View vulnerability issues](#)
[View All Vulnerability Findings](#)
[View vulnerable assets](#)
[Vulnerability Intelligence](#)
[Recast CVSS scores and CVSS severities](#)
[Reference and developer docs](#)

Vulnerability policies



A vulnerability policy defines the action you want to take for a specific set of vulnerability findings that match your policy criteria. Cortex XDR provides a set of predefined vulnerability policies based on CVSS severity, EPSS severity, and vulnerabilities confirmed through Attack Surface Testing. You can also create custom policies based on your unique business requirements. Custom policies allow you to focus on the risks that matter most to your organization. Some examples of custom vulnerability policies include the following:

- A policy that creates issues with a severity of **critical** for findings that have a CVSS score of 9 or more
- A policy that creates issues with a severity of **low** for findings that appear on dev servers
- A policy that specifies not to create issues for findings on assets in the asset group **Leased to customers**
- A policy which creates issues with a severity of **critical** for vulnerabilities that appear on the CISA KEV list **and** are in the asset group called **Production Servers**, regardless of CVSS score.
- A policy that prevents an image that contains code with a CVE with an EPSS score greater than 90% from being deployed to the Kubernetes cluster

Each time a new vulnerability finding is discovered, the system compares that finding to your vulnerability policies to determine whether one of the policies is a match. Vulnerability policies have an evaluation order, which means the system starts by evaluating the finding against the first policy. If it does not match, the second policy is evaluated for a match. As soon as a finding matches a policy, no further policies are evaluated for that finding.

The following sections describe the elements that make up a vulnerability policy:

[Policy conditions and scope](#)

[Policy actions](#)

[Policy order](#)

Create a vulnerability policy



Before creating a policy, be sure to review the information in the [Vulnerability Policies](#) section.

1. Navigate to Posture Management → Rules & Policies → Policies → Vulnerability Management.
2. Click +Add Policy and select one of the options:
 - Create a policy for issue creation
 - Create a policy for prevention
3. Add a Policy Name and, optionally, a Description, and then click Next.
4. Set the policy conditions by creating a query that defines the specific findings for which the policy will create issues. Your policy can specify which findings to include and which to exclude.

Preview the list of findings that match your policy. If the results look correct, click Next.

5. Define the policy scope by selecting one or more asset groups from the dropdown menu. If you don't choose an asset group, the policy will apply to all assets.

If you want to create a new asset group, click Create New Asset Group to open the Asset Groups page in a new browser tab. Click + Add Group and follow the instructions in the wizard. After you've created the new asset group, go back to your original tab and finish creating your policy with new asset group.

Click Next.

6. Choose the action that will be executed on the findings that match the policy. If you select Create an issue for each matching finding, you must also select the issue severity that will be applied to those issues. You can base the severity of the issue on the severity of the underlying CVE by selecting Use Default CVE Severity in the dropdown menu.
7. Click Done.

The policy wizard will close, and you will be redirected back to the Vulnerability Policies page.

8. Set the order of evaluation for the policy.

By default new policies are added to bottom of the policy list. To move a policy up or down in the list, click and hold the arrows in the Name column and drag the policy to the desired position in the list.

We recommend placing wider-reaching, more generic policies towards the bottom of the policy list, and more specific policies towards the top of the list.

Click Save.

Update the Ignored CVEs, Asset Groups, and Assets policy



Policy number 0 in the policy list is the Ignored CVEs, Asset Groups, and Assets policy. This policy contains a list of vulnerabilities and assets for which Cortex XDR will not create vulnerability issues. Findings will still be created for these vulnerabilities and assets, and you can review those on the Vulnerabilities and Vulnerable Assets pages. You can update the Ignored CVEs, Asset Groups, and Assets policy at any time by using the following steps to add or remove assets, asset groups, and CVEs.

1. Navigate to Posture Management → Rules & Policies → Policies → Vulnerability Management.
2. The first policy in the policy list is the Ignored CVEs, Asset Groups, and Assets policy. Click on that policy to open the policy wizard.
3. Add or remove vulnerabilities, asset groups, and assets as needed. Click Next.
 - To add CVEs, asset groups, or assets, use the search bar in each section to find the value you are looking for, and select it to add it to the list.
 - To remove CVEs, asset groups, or assets, click the X to the right of each item in the list.
4. Review the Results Preview to see the list of findings that will not generate issues. If the list looks correct, click Done.

Modify a vulnerability policy



1. Navigate to Posture Management → Rules & Policies → Vulnerability Management.
2. Select either the Issue Creation or Prevention tab, depending on the type of policy you want to modify.
3. Click on the name of the policy in the policy list to open the policy wizard. You can also right-click anywhere in the row and select Edit.
4. Follow the steps in the wizard to update the policy.

Configure a block grace period



When you create a Vulnerability Management Prevention policy, you also have the option to establish a remediation buffer period. Configuring a block grace period gives you additional time to resolve a vulnerability before the blocking action resumes. The grace period is based on the fix date of the vulnerability and allows you to override the blocking action of a policy when new vulnerabilities are detected. Follow the steps below to set up a block grace period:

1. Navigate to Posture Management → Rules & Policies → Vulnerability Management → Vulnerability Policies - Prevention.
2. Select an existing policy or create a new policy with the Add Policy button.
3. Add a Policy Name, Optional Description, and click Next.
4. Set the Policy Conditions and Policy Scope, as described under [Create a vulnerability policy](#).
5. Select an Action that will be triggered when a finding matches the policy.
6. For Kubernetes Runtime protection, if you opt to Prevent new deployment requests, you can also select a block grace period, during which the preventive action will be suppressed. Enter a value in the Grace Period Days before Blocking Deployment field. The grace period begins on the fix publish date, or the date the vulnerability was published if a fix is not available. Blocking enforcement begins once the grace period has passed. Enter 0, to immediately start blocking action.
7. For Prevention Actions, if you opt to Fail the build, you can also select a block grace period, during which the preventive action will be suppressed. Enter a value in the Grace Period Days before Failing the Build. Select Done to save your changes.

Enable or disable a vulnerability policy



After disabling a policy, no new issues will be created or actions taken for new findings that match the policy.

1. Navigate to Posture Management → Rules & Policies → Vulnerability Management.
2. Select either the Issue Creation or Prevention tab, depending on the type of policy you want to modify.
3. Right-click anywhere in the row for that policy and select Enable or Disable.

Investigate and remediate vulnerabilities



Cortex XDR provides several ways to view and track vulnerability data so you can monitor, investigate, and remediate vulnerabilities in your environment.

View all Vulnerabilities





The Vulnerabilities page displays all your vulnerabilities grouped by CVE or other vulnerability ID. This view shows you how prevalent each vulnerability is in your environment. The Vulnerabilities page includes key information about each vulnerability, with links to the related lists of instances (also called findings), related issues, and impacted assets.

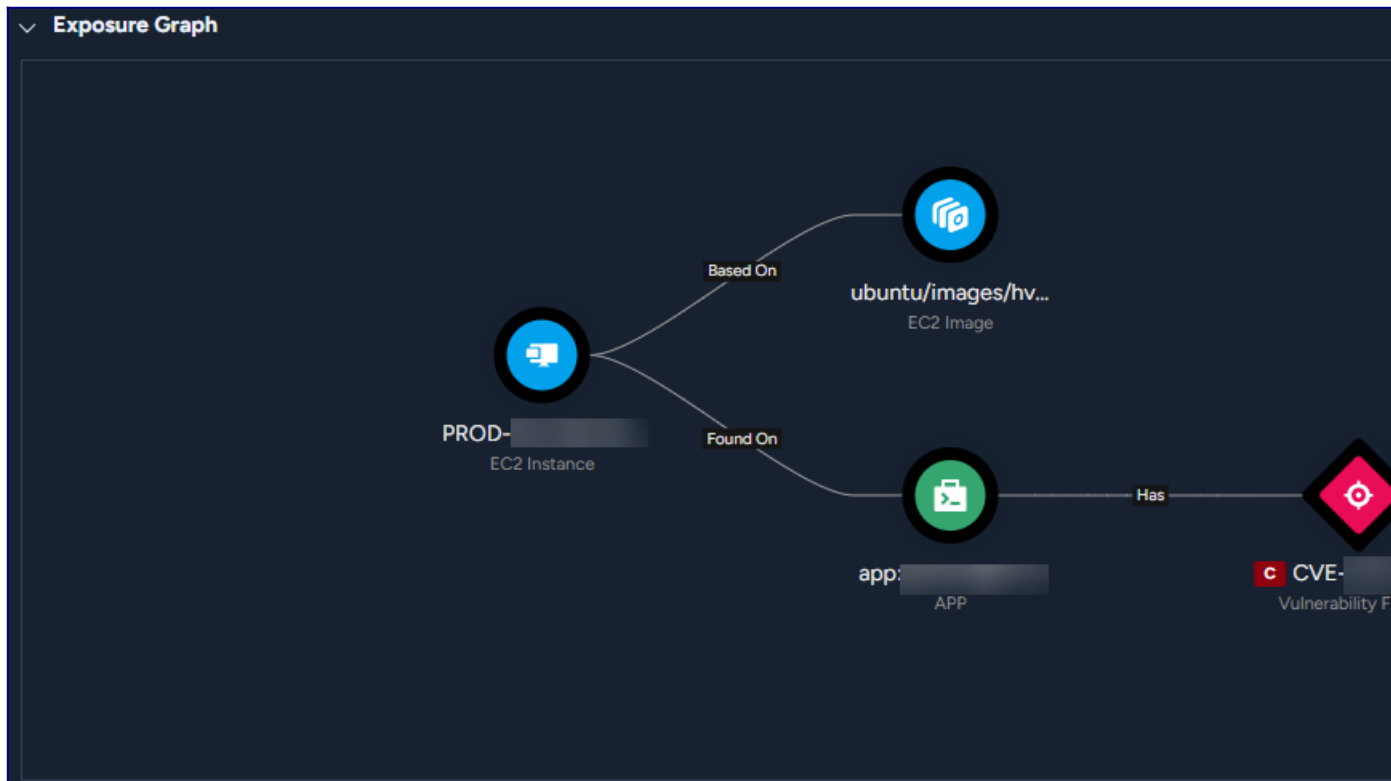
Go to Posture Management → Vulnerability Management → Vulnerabilities.

View vulnerability issues



The Vulnerability Issues page displays all vulnerability issues along with critical vulnerability intelligence and context so you can assign an issue to an owner, investigate, remediate, and track progress. Follow the steps below to view Vulnerability Issues and investigate them further:

1. Go to Posture Management → Vulnerability Management → Vulnerability Issues. Click on an issue in the table to display the issue details panel. You can also optionally generate a .tsv file export of the data if required from the Vulnerability Issues page.
2. The Overview tab on the Vulnerability Issues panel captures all the relevant details to further investigate the vulnerability including Summary, Vulnerable Asset and Code Details, Security Controls, and Remediation Steps. In addition, you can further isolate the vulnerability by examining the Exposure Graph. Use the graph to investigate relationships between K8s Namespaces, Clusters, Workloads, and Container Images.



3. Review the Exposure Graph to understand where the vulnerability originates and how it impacts Findings and Assets. Select More Info to view granular details about each of the impacted assets.
4. You can also select View Details from the Details view to be directed to the Finding or Asset in question.

View All Vulnerability Findings



A vulnerability finding is a specific instance of a vulnerability that was discovered in your environment. The All Vulnerability Findings page lists every instance of every vulnerability that was discovered in your environment.

Go to Posture Management → Vulnerability Management → Vulnerability Issues and click the All Vulnerability Findings button.

View vulnerable assets



The Vulnerable Assets page displays all assets with a vulnerability finding. This view enables you to prioritize vulnerabilities by asset and asset type and focus on assets most critical to fix. The Vulnerable Assets list provides links to the findings and issues for each asset. Click on an asset in the table to see the asset details.

Go to Posture Management → Vulnerability Management → Vulnerable Assets.

Vulnerability Intelligence



Vulnerability Intelligence is a real-time feed that contains vulnerability data and threat intelligence from a variety of certified upstream sources. This feed continuously pulls data from known vulnerability databases, official vendor feeds and commercial providers to provide the most accurate vulnerability detection results.

In addition to the information collected from official feeds, Vulnerability Intelligence is enriched with data curated by a dedicated research team. Our security researchers monitor cloud and open-source projects to identify security issues through automated and manual means. As a result, we can detect new vulnerabilities that were only recently disclosed, and even vulnerabilities that were quietly patched.

Vulnerability Intelligence provides comprehensive, actionable information including the following:

- CVE metadata, such as description, impact, severity, and CVSS v2/3/4 Scores
- CPE and product information, such as affected packages, versions, and OS

- Exploit intelligence, such as Exploit Availability, maturity, exploitability and EPSS scores
- Links to vendor advisories

How to view Vulnerability Intelligence

1. Navigate to Posture Management → Vulnerability Management → Vulnerability Intelligence.
2. (Optional) Click a row in the Vulnerability Intelligence table to view detailed information about the vulnerability.

The details page has an Overview tab, which provides information about the vulnerability and an Affected Software tab, which shows information about all the software packages impacted by the vulnerability.

Recast CVSS scores and CVSS severities



In some situations, you might decide that a specific vulnerability poses a different level of risk to your environment than what is reflected in the original CVSS score or CVSS severity. In Cortex XDR you can override the CVSS score or severity within the platform. Customizing CVSS scores and severities enables you to align your risk management approach with your unique context and priorities.

When a CVSS score or severity is recast, the change is applied platform-wide, updating both existing and new vulnerability findings. This ensures consistency in how vulnerabilities are assessed and managed across the organization. After the CVSS score or severity is updated, the system automatically updates all affected findings within about one hour.

You can view the original CVSS score and severity and new values on the vulnerability details page in Vulnerability Intelligence.

How to recast the CVSS score and CVSS severity of a vulnerability

1. Navigate to Posture Management → Vulnerability Management → Vulnerability Intelligence.
2. Use the filters to find the vulnerability in the Vulnerability Intelligence table.
3. Click in the row for the vulnerability to open the vulnerability details panel.
4. Click the Options icon in the upper right corner and select Override Severity or CVSS.
5. Enter the new severity and score, and then click Save.

Reference and developer docs



Cortex XDR XQL



Understand more about the Cortex Query Language called XQL, so you can build queries to gain insight from the data contained in the different data sources in Cortex XDR.

Get started with XQL



XQL is the Cortex Query Language. It allows you to form complex queries against data stored in Cortex XDR. This section introduces XQL, and it provides reference information on the various stages, functions, and aggregates that XQL supports.

XQL language features



The Cortex Query Language (XQL) enables you to query for information contained in a wide variety of data sources in Cortex XDR for rigorous endpoint and network event analysis. Queries require a dataset, or data source, to run against. Unless otherwise specified, the query runs against the `xdr_data` dataset, which contains all raw log information that Cortex XDR collects from all Cortex product agents, including EDR data, and PAN NGFW data. You can also import data from third parties and then query against those datasets as well.

You submit XQL queries to Cortex XDR using the Investigation & Response → Search → Query Builder user interface.

XQL is similar to other query languages, and it uses some of the same functions as can be found in many SQL implementations, but it is not SQL. XQL forms queries in stages. Each stage performs a specific query operation and is separated by a pipe (|) character. To help you create an effective XQL query with the proper syntax, the query field in the user interface provides suggestions and definitions as you type. For example, the following query uses three stages to identify the dataset to query, identify the field to be retrieved from the dataset, and then set a filter that identifies which records should be retrieved as part of the query:

```
dataset = xdr_data
| fields os_actor_process_file_size as osapfs
| filter to_string(osapfs) = "12345"
```

When creating XQL queries, you can:

- Use the up and down arrow keys to navigate through the auto-suggestion commands and definitions.
- Select an auto-suggestion command by pressing either the Enter or Tab key.
- Press Shift+Enter to add a new line, and easily ignore the auto-suggestion output.
- Close the auto-suggestion output by pressing the Esc key.

XQL supports:

- Simple queries.
- Filters that identify a subset of records to return in the result set.
- Joins and Unions.
- Aggregations.
- Queries against standard datasets.
- Queries against presets, which are collections of information that are specific to a given type of network or endpoint activity, such as authentication or file transfers.
- Queries against custom imported datasets.

XQL Language Structure



Cortex Query Language (XQL) queries usually begin by defining a data source, be it a dataset or a preset. After that, you use zero or more stages to form the XQL query. Each stage is delimited using a pipe (|). The function performed by each stage is identified by the stage keyword that you provide.

Specifying a dataset is not required because Cortex XDR uses `xdr_data` as the default dataset. If you have more than one dataset or lookup, you can change your default dataset by navigating to Settings → Configurations → Data Management → Dataset Management, right-click on the appropriate dataset, and select Set as default.

In the simplest case, you can specify a dataset using one of the following formats.

- Hot Storage queries are performed on a dataset using the format `dataset = <dataset name>`. This is the default option.

```
dataset = xdr_data
```

- Cold Storage queries are performed using the format `cold_dataset = <dataset name>`.

```
cold_dataset = xdr_data
```

You can also build a query that investigates data in both a cold dataset and hot dataset in the same query. In addition, as the hot storage dataset format is the default option and represents the fully searchable storage, this format is used throughout this guide. For more information on hot and cold storage, see [Dataset management](#).

When using the hot storage default format, this returns every `xdr_data` record contained in your Cortex XDR instance over the time range that you provide to the Query Builder user interface. This can be a large amount of data, which might take a long time to retrieve. You can use a `limit` stage to specify how many records you want to retrieve.

```
dataset = xdr_data | limit 5
```

The records resulting from this query, or the result set, are returned in unsorted order. Every time you run the query, it will probably return a different set of records in no specific order. To create a predictable result set, use other stages to define [sort](#) order, [filter](#) the result set to identify exactly what records you want returned, to [create fields containing aggregations](#), and more.

There is no practical limit to the number of stages that you can specify. See [Stages](#) for information on all the supported stages.

In the `xdr_data` dataset, every user field included in the raw data, for network, authentication, and login events, has an equivalent normalized user field associated with it that displays the user information in the following standardized format:

```
<company domain>\<username>
```

For example, the `login_data` field has the `login_data_dst_normalized_user` field to display the content in the standardized format. We recommend that you use these `normalized_user` fields when building your queries to ensure the most accurate results.

Adding comments in queries



You can add comments in any section when building a query in Cortex Query Language (XQL).

- Comments are added on a single line using the following syntax.

```
//<comments>
```

For example,

```
dataset = xdr_data
| filter event_type=1
//ENUM.process
and event_sub_type = 1
//ENUM.execution
```

- To write a comment that extends over multiple lines use the following syntax.

```
/*multi-line <comments> */
```

For example,

```
dataset = xdr_data
| filter
/*multi-line Adding comments is a great thing.
Here is an example */
event_type=1
```

Supported operators



Cortex Query Language (XQL) queries support the following comparison, boolean, string, range, and add operators.

Operator	Description
Comparison operators	
=, !=	Equal, Not equal
<, <=	Less than, Less than or equal to
>, >=	Greater than, Greater than or equal to
Boolean operators	
and	Boolean and
or	Boolean or

Operator	Description
not	Boolean not
String and range operators	
	Returns true if the integer or string field value is one of the options specified. For example:
	<code>action_local_port in(5900,5999)</code>
IN, NOT IN	For string field values, wildcards are supported. In this example a wildcard (*) is used to search if the value contains the strings "word_1" or "word_2" anywhere in the output, or exactly matches the string "word":
	<code>str_field in ("*word_1*", "*word_2*", "word")</code>
	In some cases, using an IN or NOT IN operator combined with a dataset and filter stage can be a better alternative to using a join stage.
CONTAINS, NOT CONTAINS	Performs a search for an integer or string. Returns true if the specified string is contained in the field. <code>Contains</code> and <code>Not Contains</code> are also supported within arrays for integers and strings.
	<code>lowercase(actor_process_image_name) contains "psexec"</code>
~=	Matches a regular expression.
	<code>action_process_image_name ~= ".*?\.(?:pdf docx)\.exe"</code>

Operator	Description
INCIDR, NOT INCIDR	Performs a search for an IPv4 address or IPv4 range using CIDR notation, and returns true if the address is in range. <pre>action_remote_ip incidr "192.1.1.1/24"</pre>
	It is also possible to define multiple CIDRs with comma separated syntax when building a XQL query with the Query Builder or in Correlation Rules. When defining multiple CIDRs, the logical OR is used between the CIDRS listed, so as long as one address is in range the entire statement returns true. The same logic is used when using the <code>incidr()</code> function. For more information on how this logic works to determine whether the <code>incidr</code> or <code>not incidr</code> operators return true or false, see incidr. <pre>action_remote_ip incidr "192.168.0.0/24, 1.168.0.0/24"</pre>
	Both the IPv4 address and CIDR ranges can be either an explicit string using quotes (""), such as "192.168.0.1", or a string field. Performs a search for an IPv6 address or IPv6 range using CIDR notation, and returns true if the address is in range. <pre>action_remote_ip incidr6 "3031:3233:3435:3637:0000:0000:0000:0000/64"</pre>
INCIDR6, NOT INCIDR6	It is also possible to define multiple CIDRs with comma separated syntax when building a XQL query with the Query Builder or in Correlation Rules. When defining multiple CIDRs, the logical OR is used between the CIDRS listed, so as long as one address is in range the entire statement returns true. The same logic is used when using the <code>incidr6()</code> function. For more information on how this logic works to determine whether the <code>incidr6</code> or <code>not incidr6</code> operators return true or false, see incidr6. <pre>action_remote_ip incidr6 "2001:0db8:85a3:0000:0000:8a2e:0000:0000/64, fe80::/10"</pre>
	Both the IPv6 address and CIDR ranges can be either an explicit string using quotes (""), such as "3031:3233:3435:3637:0000:0000:0000:0000/64", or a string field.

Add operator for tagging

Operator

Description

The `add` operator is used in combination with the `tag` command to add a single tag or list of tags to a field that you can easily query in the dataset.

- Adding a Single Tag

`add`

```
dataset = xdr_data  
| tag add "test"
```

- Adding a List of Tags

```
dataset = xdr_data  
| tag add "test1", "test2", "test3"
```

Datasets and presets



Every Cortex Query Language (XQL) dataset query begins by identifying a data source that the query will run against. Each data source has a unique name, and a series of fields. Your query specifies the data source, and then provides stages that identify fields of interest and perform operations against those fields.

You can query against either datasets or [Presets](#) in a dataset query. XQL supports using different languages for dataset and field names. In addition, the dataset formats supported are dependent on the data retention offerings available in Cortex XDR according to whether you want to query hot storage (default) or cold storage. For more information, see [XQL Language Structure](#).

[Datasets](#)

[Presets](#)

About examples



The examples included in the topics are intended to illustrate the behavior or usage of a particular stage or function. While these examples can be based on real data that you could use on real-world queries, you may need to tweak these queries to perform investigations or otherwise solve real-world problems.

For examples of queries that illustrate useful investigative queries, see the example Query Library that is available from the product user interface:

Investigation & Response → Search → Query Builder → XQL → Query Library

JSON functions



The Cortex Query Language (XQL) includes a number of JSON functions. Before using any of these functions, it's important to understand how Cortex XDR treats a JSON so you can accurately formulate your queries using the correct syntax.

JSON field names are case sensitive, so the key to field pairing must be identical in an XQL query for results to be found. For example, if a field value is "TIMESTAMP" and your query is defined to look for "timestamp", no results will be found.

[<json_path>](#)

[Field in <json_path> contains characters](#)

How to filter for empty values in the results table





When building a query, you can filter for empty values in the results table, which can include or exclude null or empty strings. In the query syntax, empty strings are represented as "", while null fields are represented as null.

- Exclude null and empty strings using the following syntax:

<name of field> != null and <field name> != ""

- Include null or empty strings using the following syntax:

<name of field> = null or <field name> = ""

Below is an example of filtering your endpoint data in the results table to exclude all null values and any empty strings for a user.

```
config timeframe = 90d
| dataset = endpoints
| filter endpoint_status in (CONNECTED, DISCONNECTED)
| filter user != null and user != ""
| fields user, group_names, endpoint_name
```

Understanding string manipulation in XQL



When defining string fields in Cortex Query Language (XQL) queries, it's important to understand the various string manipulations available and the syntax required to build effective queries that return the results you're expecting. Cortex Query Language (XQL) uses [RE2](#) for its regular expression implementation.

Cortex XDR enables you to use single double quotes ("<text>") or triple double quotes ("<text>") when defining your XQL syntax for string manipulation. This specific syntax is used with different stages, functions, and operators, with or without wildcards. Typically, the `alter` and `filter` stages are used with single or triple double quotes, so these stages are used in the examples provided below.

[Using single double quotes](#)

[Using triple double quotes](#)

[Query example using alter](#)

[Query example using filter](#)

Build XQL queries



To support investigation and analysis, you can search your data by creating queries in the Query Builder.

If you have the [Cortex Agentic Assistant](#), you can use natural language prompts to create and run XQL queries within the chat interface. For more information, see [Create and run XQL queries with Agentic Assistant chat](#).

About the Query Builder



The Query Builder aids in the detection of threats by allowing you to search for indicators of compromise and suspicious patterns within data sources. It assists in expanding case investigations by identifying related events and entities, such as activities associated with specific user accounts or network lateral movement. In addition, the Query Builder enables data analytics on suspected threats, helping organizations analyze large volumes of data to identify trends, anomalies, and correlations that may indicate potential security issues. The Query Builder also provides an interactive and visually intuitive way for you to search assets and findings by their relationship types and map them out in real-time.

To support investigation and analysis, you can search all of the data ingested by Cortex XDR by creating queries in the Query Builder. You can create queries that investigate leads, expose the root cause of an issue, perform damage assessment, and hunt for threats from your data sources.

Cortex XDR provides different options in the Query Builder for creating queries:

- XQL (Build your own queries)

You can use the Cortex Query Language (XQL) to build complex and flexible queries that search specific datasets or presets, or the entire `xdr_data` dataset. With XQL Search, you create queries based on stages, functions, and operators. To help you build your queries, Cortex XDR provides tools in the interface that provide suggestions as you type, or you can look up predefined queries, common stages and examples. For more information, see [How to build XQL queries](#).

Schema changes to datasets may not be reflected in the autocomplete suggestions and definitions as you type in real time the XQL query, and can appear with a slight delay.

When creating XQL queries, you can:

- Use the up and down arrow keys to navigate through the auto-suggestion commands and definitions.
 - Select an auto-suggestion command by pressing either the Enter or Tab key.
 - Press Shift+Enter to add a new line, and easily ignore the auto-suggestion output.
 - Close the auto-suggestion output by pressing the Esc key.
- Predefined queries for different types of entities
 - Graph Search to build queries to search assets, findings, and their contextual data. For more information, see [How to build Graph Search queries?](#).

[Discover more](#)

[Cortex Cloud Identity Security](#)

[Application Security Posture Management \(ASPM\)](#)

[Web and API Security \(WAAS\)](#)

[Serverless Function Runtime Security](#)